



山东大学
SHANDONG UNIVERSITY

WaterOS 初赛设计文档

参赛队名: OuterSystems

队伍成员: 宋浩宇、李佳灿、孙馨宇

指导老师: 颜廷坤、潘润宇

目 录

第 1 章 项目概述	1
1.1 设计目标	1
1.2 当前实现摘要	2
1.3 当前局限	3
第 2 章 总体架构设计	4
2.1 组件化分层	4
2.2 代码结构	4
2.3 组件目录与职责映射	5
2.4 API 与实现解耦	6
2.5 启动主线	6
2.6 Feature 组合与平台构建	7
2.7 QEMU 双架构适配与一致性	13
2.8 内核入口代码流程	13
第 3 章 模块实现	15
3.1 内核集成：异常与系统调用入口	15
3.2 ABI 组件	16
3.3 基础类型组件	16
3.4 凭证组件	17
3.5 设备驱动组件	18
3.6 文件系统组件	18
3.7 进程间通信组件	19
3.8 内核日志组件	19
3.9 内存管理组件	20
3.10 平台组件	21
3.11 运行时组件	23
3.12 系统调用组件	24
3.13 任务管理组件	24
3.14 工具组件	25
3.15 虚拟文件系统组件	26
第 4 章 测试与问题处理	27
4.1 构建与启动方式	27
4.2 功能验证	28
4.3 双架构一致性验证	29
4.4 遇到的问题和解决方法	30

第 5 章	总结与后续工作	36
5.1	工作总结与项目特色	36
5.2	后续完善方向	36
5.3	非本队来源说明	37
5.4	AI 工具使用说明	38

第 1 章 项目概述

WaterOS 是用 Rust 编写的 `no_std` 操作系统内核，面向操作系统竞赛在 QEMU 上完成 bring-up 与测例验收。内核已在 RISC-V64 OpenSBI 与 LoongArch64 virt 平台两条主线上跑通启动、trap 分发、分页、任务调度与用户 ELF 执行；文件系统、VFS、系统调用和 IPC 也接在同一套用户态路径上。

两条平台路径的入口参数、trap frame、页表格式、定时器与 VirtIO 总线不同，差异收束在 `wateros-platform`、`wateros-driver` 和 `wateros-mm` 各自的 `impl-* crate` 中。进入 task、FS/VFS、IPC、cred、klog 和 syscall 之后，上层逻辑共用同一套接口，用户程序在两条路径上看到的 fd、进程、地址空间与系统调用编号保持一致。

立项时除了竞赛验收，我们也希望内核能支撑多人并行开发和教学演示。单体仓库里改一处驱动往往要牵一串头文件和全局状态；我们更想要的是：组员互不了解对方模块细节，仍能在抽象层上对接。熟悉某一域的人把 `api-v0` 里的 trait 和类型定下来，再用本层原语往上拼接口；只打算贡献某一个 `impl-*` 的人可以在 `crate` 内部实现细节，不必追问这些原语最后被谁、以何种方式调用。组装操作系统时因此主要面对稳定模块名和契约，而不是同时盯着所有底层实现。

Rust 和这种拆法比较合拍。`api-v0` 用 trait 列出必须实现的行为，pub 边界控制后来接手的人能看到的面；所有权和借用检查把不少内存与并发问题拦在编译期，代码评审时可以把精力放在逻辑是否正确、feature 组合是否合理。每个 `impl-*` 在 `Cargo.toml` 里注册成 feature 之后，根 `crate` 切换 feature 即可换实现：驱动允许多个 `impl-*` 通过 feature 并存；堆分配器、平台 arch 实现、调度器策略这类只能二选一或 N 选一的，则在聚合 `lib.rs` 里用 `compile_error!` 在配置冲突时直接报错，例如 `runtime-heap-allocator` 要求 `impl-tlsf` 与 `impl-linked-list-allocator` 不能同时启用。

竞赛阶段先把 QEMU bring-up 跑通；更长远的打算是同一套边界还能留给实验性文件系统、网络栈或教学用小内核。扩展留在 feature 和 `impl` 层，换板子或换算法时不必每次从 `kernel_main` 重新捋全局状态。根 `crate` `wateros` 按 feature 聚合一级组件，各组件再由 `lib.rs` 选出当前 active `impl` 后对外只暴露 `mm::`、`vfs::`、`ipc::` 等稳定导出名。

1.1 设计目标

- 在 QEMU 中完成可启动、可抢占调度、可装载并执行用户 ELF 的内核主线。
- 系统调用编号与参数布局对齐 Linux generic 64-bit ABI，使赛题根卷内的 `glibc/musl` 测程可直接 `ecall` 进入内核。
- 在 RISC-Vvirt 与 LoongArchvirt 上分别接通 VirtIO 块设备、ext4 读写根卷与 VFS fd

会话，支撑 busybox 脚本链与 network benchmark。

- 保持平台相关代码与 MM、task、VFS、syscall 等上层模块分离，新增板级或 ISA 时只替换底层 impl-*, 不改动上层聚合接口。

1.2 当前实现摘要

表 1-1: WaterOS 当前实现摘要

子系统	当前实现状态
启动与平台	默认 feature 为 qemu-riscv64-opensbi, 第二主线为 qemu-loongarch64-virt。RISC-V 入口从 OpenSBI 接收 hart id 与 DTB 物理地址; LoongArch 入口接收 argc/argv/envp, 建内核页表前关闭固件遗留的分页。kernel_main 顺序为 runtime/platform 初始化、MM 与 task/trap、驱动 init_after_boot、fs::init 探测块设备但不挂载、用户 bring-up 总线、定时器、task::run_first_task。
内存管理	RISC-V 侧为 Sv39, LoongArch 侧为独立三级 4KiB 页表; 物理帧分配默认 impl-stack, 采用栈式 LIFO 策略。已接通内核页表、用户地址空间、ELF 装载、COW fork、惰性 mmap、用户缺页、user_access 拷贝, 以及 brk、mmap、munmap、mprotect 等内存类系统调用。
任务与调度	impl-core 提供内核/用户任务、fork/clone/exec、wait 族与 zombie 回收; bring-up 以 SCHED_OTHER 与 timer tick 调度为主。多类调度队列已含 FIFO/RR 骨架, 当前有效策略仍为 CFS 类路径; 单核假设, 阻塞与睡眠委托 waitqueue。
驱动	RISC-V 经 DTB 扫描 VirtIO-MMIO 块设备与网卡; LoongArch 经 PCI 枚举 VirtIO-PCI 设备。块设备注册到统一 BlockDevice 表并配合 impl-block-cache; 网络栈为 smoltcp, 字符设备含 UART 与若干桩节点; 非 QEMU 板级仅有 impl-dummy 占位。
文件系统与 VFS	默认 impl-ext4-rs 提供 ext4 读写根卷, 挂载入口为 mount_default_root_rw; devfs、procfs 与根卷句柄由 impl-kernel 实现。VFS 经 impl-fs-bridge 消费 FS API, 叠加挂载表、页缓存与 impl-fd-session 的 per-task fd 表、cwd、dup/fork 继承与 CLOEXEC; FsAsyncIo 尚未实现。

系统调用	syscall/impl-kernel 覆盖文件 IO、进程、内存、时间、目录、凭证、pipe、poll/epoll、futex、SysV shm、INET/UNIX socket 子集与 syslog；号表来自 abi/impl-linux-generic64，错误统一为-errno。未实现槽位在 bring-up 期可能 panic；ioctl 仅覆盖 TTY 等子集。
IPC 与信号	根 feature 启用 ipc/all: waitqueue、ringbuf pipe、futex 由 impl-task 实现，SysV shm 子集与 signal 状态机已进入主路径。聚合层 wateros-ipc-api-v0 仍为占位自检；ipc-event 未挂接。信号覆盖 disposition、pending、mask、signal frame 与 syscall restart 的基础路径。
验证与复现	user_bringup_bus 挂载 RW ext4 根卷与 procfs 后，默认串行执行 stage-busybox 脚本队列。已在双 libc 路径上跑通 basic、busybox、lua、iperf、netperf、libctest、cyclicttest、libcbench、iozone、lmbench 等赛题脚本；LTP 仅覆盖发布测例的子集。另有 stage-basic、stage-02-mm、POSIX FS 烟囱等入口，可按需单独打开；启动日志含 MM、驱动、FS、VFS 与网络栈分层自检。

1.3 当前局限

本节记录与赛题验收相关、但尚未达到完整 Linux 语义或生产可用的缺口；细节以各组件 docs/exports/features/快照为准。

- **单核与中断**。当前为单 hart bring-up，无 SMP 调度，也无完整 IRQ 驱动链；抢占依赖定时器 tick 与显式 yield/block。
- **Linux 语义子集**。调度 RT 策略、NUMA、完整/proc 字段、SUID exec、journal 恢复等均未按生产内核实现；procfs 与 securityfs 以满足 LTP 子集为主。
- **凭证与安全**。wateros-cred 的 cap 与 inode 权限多为桩；VFS open 未与凭证深度联动；klog 的 syslog 读写未做权限校验，且无/dev/kmsg 节点。
- **stub 与占位 crate**。wateros-ipc-api-v0 与部分 impl-dummy 仅保证编译通过；非 QEMU 平台无法真实启动。
- **LTP 覆盖**。ltp_testcode.sh 已接入 bring-up 队列，但内核 syscall/VFS/信号语义未覆盖 LTP 全集，部分用例预期失败或需从队列中排除。
- **用户态工具链**。仓库内 user/子模块仅构建 RISC-V 用户态烟测 ELF；LoongArch 测程依赖根卷内预置的 glibc/musl 二进制与脚本。

第 2 章 总体架构设计

2.1 组件化分层

WaterOS 按依赖方向分成三层：底层接平台和硬件，中层管内核资源，上层给用户态可见的 ABI 与系统调用语义。RISC-V64 与 LoongArch64 的差异留在 platform、driver 和 mm 各自的 impl-* 里；task、fs、vfs、ipc 和 syscall 等上层模块共用同一套聚合接口。

表 2-1: WaterOS 组件化分层

层次	主要组件	设计含义
平台与运行时层	platform、runtime、driver	启动入口、trap 汇编、定时器、中断、固件调用、控制台、开发日志、堆和设备发现；把 QEMU 与 ISA 差异收成内核内部接口。
内核资源层	mm、task、fs、vfs、ipc	地址空间、物理页、任务生命周期、调度、根文件系统、路径视图、fd 表、pipe、waitqueue、futex 和 signal 等核心资源。
用户接口层	abi、syscall、cred、klog	Linux generic 64-bit 系统调用编号、参数传递、错误码、凭证扩展点，以及 syslog 可读的内核日志环。

根 cratewateros 按 feature 聚合一级组件；各组件内部再拆 api-v0、impl-* 和聚合 lib.rs。契约写在 API crate，实现落在 impl crate，聚合层按 feature 选出当前 active impl 后对外只暴露稳定模块名，例如 mm::、vfs::、ipc::。目录树与下表从源码布局与模块边界两个角度描述同一套组织方式。

2.2 代码结构

内核源码主体在 os/；仓库根 Makefile 封装常用构建入口。平台配置、feature 开关、脚本和组件实现都收在 os/内，由根 Cargo.toml 统一聚合。

```
wateros
|-- Makefile           # 顶层构建入口
`-- os                 # 内核源码与构建主体
    |-- Cargo.toml     # 根 crate、feature 与 workspace 配置
```

```

|  |-- Makefile                # 内核构建与运行入口
|  |-- build.rs                # 构建脚本
|  |-- config.conf             # 当前启用的 feature 选择 (可编辑)
|  |-- feature-tree.txt        # workspace 全量 feature 树 (只读参考)
|  |-- src
|  |   |-- main.rs             # kernel_main 与平台启动主线
|  |   |-- trap_handler.rs     # trap/syscall 入口桥接
|  |   |-- user_bringup_*.rs   # 用户程序与测试脚本 bring-up
|  |   `-- self_tests/         # 内核自检入口
|  |-- components              # WaterOS 组件化内核模块
|  |   |-- wateros-base        # 地址类型、同步原语等基础定义
|  |   |-- wateros-platform    # 平台、架构、固件抽象
|  |   |-- wateros-runtime     # 控制台、日志、panic、堆
|  |   |-- wateros-mm          # 页表、帧分配、ELF 装载
|  |   |-- wateros-task        # 任务、调度、等待与生命周期
|  |   |-- wateros-driver      # DTB/PCI、VirtIO、块/字符/网络设备
|  |   |-- wateros-fs          # devfs、rootfs、ext4、procfs
|  |   |-- wateros-vfs         # 路径视图、fd 表、cwd、页缓存
|  |   |-- wateros-syscall     # 系统调用分发与实现
|  |   |-- wateros-abi         # errno、返回值、syscall 编号
|  |   |-- wateros-ipc         # pipe、waitqueue、futex、signal
|  |   |-- wateros-cred        # uid/gid 与凭证生命周期
|  |   `-- wateros-klog        # 内核日志环与 syslog(2)
|  |-- scripts                 # QEMU、GDB、构建和日志脚本
|  `-- vendor                   # 离线依赖源码

```

os/src/main.rs 负责启动顺序与组件编排；具体能力由 os/components/下各 crate 提供。根 crate 本身只做 feature 组合、trap 桥接和 bring-up 总线挂载。

2.3 组件目录与职责映射

表 2-2: 组件目录与职责映射

目录	职责说明
wateros-platform	启动参数、trap、中断、时间、固件调用和关机路径；RISC-V64 对接 QEMUvirt+OpenSBI, LoongArch64 对接 QEMUvirt 与 StableCounter/CSR 平台能力。
wateros-runtime	控制台、开发日志级别、panic handler 和全局堆分配器；QEMU 主线经 impl-platform-console 输出。
wateros-mm	物理帧、页表、内核/用户地址空间、ELF 装载，以及 brk/mmap 等内存类系统调用后端。

wateros-task	任务对象、调度队列、阻塞/睡眠/退出、父子关系、用户 trap frame 与 run_first_task 入口。
wateros-driver	按平台发现设备：RISC-V64 扫描 DTB 中的 VirtIO-MMIO，LoongArch64 枚举 VirtIO-PCI；块设备、网卡和字符设备统一注册到组件内抽象表。
wateros-fs	FsImpl 注册表、devfs、procfs、rootfs 管理和 ext4 读写实现。
wateros-vfs	路径视图、per-task fd 表、cwd、pipe/file/console 句柄和页缓存；经 impl-fs-bridge 消费 wateros-fs API。
wateros-syscall	将 trap 传入的 syscall 号和参数分发到进程、文件、内存、时间、网络等具体实现。
wateros-abi	Linux generic 64-bit 系统调用编号、errno、用户返回值和参数包等 ABI 契约。
wateros-ipc	waitqueue、pipe、futex、SysV shm、signal 等 IPC 能力，服务等待、管道、共享内存和同步阻塞。
wateros-cred	进程凭证侧表，uid/gid 读写，以及 VFS 权限检查的扩展点。
wateros-klog	内核日志环形缓冲；syslog 与 runtime-logging 分路，供启动期与运行期诊断读取。

2.4 API 与实现解耦

大多数一级组件遵循三步组织：

1. api-v0 定义稳定契约：trait、错误类型、句柄和公共数据结构。
2. impl-* 承载具体实现，例如 impl-sv39、impl-qemu-riscv64-opensbi、impl-ext4-rs。
3. 组件根 src/lib.rs 按 feature 绑定 active impl，向上再导出统一模块名。

上层代码只依赖聚合门面，不直接引用具体 impl crate。系统调用经 wateros-vfs 与 wateros-fs 公开接口访问路径和 fd；wateros-task 不反向依赖 wateros-cred，凭证继承与清理由 syscall 或 bring-up 调用点触发。

2.5 启动主线

两条 QEMU 路径的组件编排顺序一致，差异只在引导参数解析、LoongArch 建页表前关 MMU，以及驱动发现总线类型：

1. 平台汇编把控制权交给 kernel_main：RISC-V64 接收 OpenSBI 的 hart id 与 DTB 物理地址，LoongArch64 接收 argc/argv/envp，其中 envp 常作固件信息指针。
2. 初始化 runtime，console、logging、klog、heap 与 platform::arch；RISC-V64 在此后、LoongArch64 在此前分别调用 driver::init_when_boot 传入 DTB/envp 指针。

3. 在 MM 激活分页之前注册 `task::init` 与 `trap_handler::init`, 保证页故障和定时器 `trap` 有路由。
4. LoongArch64 额外调用 `platform::arch::paging::init_paging_disable_mmu`, 确认后续页表构建与 PCI ECAM/MMIO 使用直接寻址。
5. `mm::kernel_mm::init` 建立内核地址空间; 驱动成功则进入 `driver::active_impl::init_after_boot`。
6. `fs::init` 探测块设备并注入实现; `user_bringup_bus::run` 在总线内 RW 挂载 ext4 根卷并串行执行 bring-up 阶段。
7. 打开定时器与全局中断, `task::run_first_task` 把 CPU 交给就绪任务, 后续执行由 `trap/syscall` 路径驱动。

具体分页格式、DTB 保留策略、网络栈烟测和 `self_tests` 调用见第 1 章实现摘要与第 3 章模块实现; 本章只固定组件之间的先后关系。

2.6 Feature 组合与平台构建

WaterOS 用 Cargo feature 管理平台差异。根 `os/Cargo.toml` 定义 `qemu-riscv64-opensbi`, 默认与 `qemu-loongarch64-virt` 两组互斥组合; 二者都拉起完整 I/O 栈, 含 `driver`、`fs`、`ipc`、`cred`、`vfs-bridge`、`klog` 与 `syscall/impl-kernel`, 只在 `platform`、`mm` 页表实现和 VirtIO 总线上分叉。

```
[features]
default = ["qemu-riscv64-opensbi"]
impl-sv39 = ["mm/impl-sv39"]

qemu-riscv64-opensbi = [
    "abi/impl-linux-generic64",
    "dep:cred", "dep:driver", "dep:fdt", "dep:fs", "dep:ipc",
    "driver/impl-block-cache",
    "driver/impl-qemu-riscv64-opensbi",
    "ipc/all",
    "vfs-bridge",
    "impl-sv39",
    "mm/vfs-root-read",
    "platform/impl-qemu-riscv64-opensbi",
    "runtime/impl-info",
    "dep:klog", "klog/default",
    "runtime/impl-platform-console",
    "syscall/impl-kernel",
]

qemu-loongarch64-virt = [
    "abi/impl-linux-generic64",
```

```
"dep:cred", "dep:driver", "dep:fdt", "dep:fs", "dep:ipc",
"driver/impl-block-cache",
"driver/impl-qemu-loongarch64-virt",
"ipc/all",
"mm/impl-loongarch64",
"mm/vfs-root-read",
"platform/impl-qemu-loongarch64-virt",
"runtime/impl-debug",
"dep:klog", "klog/default",
"runtime/impl-platform-console",
"syscall/impl-kernel",
"vfs-bridge",
```

```
]
```

除根 Cargo.toml 里的平台组合外，组件级 api-* 与 impl-* 选择还依赖各 crate 的 default feature。make configure 会同时生成两份快照，职责不同：

- feature-tree.txt：workspace 全量 feature 树。按 package 列出所有已定义的 feature 及其向下传递链，用于查“有哪些可选组合、feature 如何传递”，不代表当前编译实际选中哪条路径。
- config.conf：当前正在使用的 feature 选择。未注释行对应当前各 Cargo.toml 里 default 已启用的项；编辑后经 make apply_features 写回 manifest，才是改组件实现的入口。

Makefile 目标见下文2.6.3。下面各举一条从根到叶子 impl 的传递示例，摘自全量树中的 RV/LA 主线：

RISC-V64 块设备链：

```
qemu-riscv64-opensbi
-> driver/impl-qemu-riscv64-opensbi          # wateros-driver
    -> block/impl-virtio-mmio                # wateros-driver-block
        -> wateros-driver-block-impl-virtio-mmio
```

同一条组合里，平台与架构链为 platform/impl-qemu-riscv64-opensbi 拉入 arch/impl-riscv64 和 firmware/impl-riscv64-opensbi；页表链为根 impl-sv39 到 mm/impl-sv39。

LoongArch64 块设备链：

```
qemu-loongarch64-virt
-> driver/impl-qemu-loongarch64-virt        # wateros-driver
    -> block/impl-virtio-pci                # wateros-driver-block
        -> wateros-driver-block-impl-virtio-pci
```

对应平台链为 `platform/impl-qemu-loongarch64-virt` 拉入 `arch/impl-loongarch64` 与 `firmware/impl-qemu-loongarch64-uart16550`；页表链为 `mm/impl-loongarch64`。两条路径的 `syscall`、`vfs`、`task` 和 `fs` 注册表 `feature` 相同，替换点集中在 `platform-arch`、`driver-impl` 和 `mm-impl`。

2.6.1 全量 feature 树，feature-tree.txt

`feature-tree.txt` 是只读参考：扫描 `workspace` 内全部 `Cargo.toml`，按 `package` 分组列出每一个已声明的 `feature`，以及启用该 `feature` 时会继续拉起的子 `crate` 与子 `feature`（子行以 `>` 指向目标 `package`）。它覆盖所有可选路径与传递链，包括当前 `config.conf` 里未启用的分支。仓库内全文件约 1700 行；下列摘自 `os/feature-tree.txt` 的代表性片段（约 50 行），用... 表示上下尚有大量 `package` 未展开：

```
# WaterOS feature tree (按 package 分组)
# root: .../os

...                                # ext4_rs、ipc-event、ipc-futex 等

ipc-pipe (components/wateros-ipc/ipc-pipe/Cargo.toml)
  - impl-ringbuf [default]
    - impl-ringbuf/impl-ringbuf
      > wateros-ipc-pipe-impl-ringbuf
        - api-v0
        - impl-ringbuf [default]

wateros (Cargo.toml)
  - default [default]
  - qemu-riscv64-opensbi
  - impl-sv39
    - mm/impl-sv39
      > wateros-mm
        - impl-sv39
        - impl-loongarch64
  - qemu-loongarch64-virt
    - mm/impl-loongarch64
      > wateros-mm ...
  - driver/impl-qemu-loongarch64-virt
    > wateros-driver ...
  - platform/impl-qemu-loongarch64-virt
    > wateros-platform ...
  - syscall/impl-kernel
    > wateros-syscall ...
  - qemu-riscv64-opensbi [default]
```

```

- driver/impl-qemu-riscv64-opensbi
  > wateros-driver ...
- platform/impl-qemu-riscv64-opensbi
  > wateros-platform ...
- ipc/all
  > wateros-ipc ...
- impl-sv39
- syscall/impl-kernel
  > wateros-syscall ...

wateros-driver (components/wateros-driver/Cargo.toml)
- impl-qemu-riscv64-opensbi
- block/impl-virtio-mmio
  > wateros-driver-block-impl-virtio-mmio
- network/impl-virtio-mmio
  > wateros-driver-network ...
- impl-qemu-loongarch64-virt
- block/impl-virtio-pci
  > wateros-driver-block-impl-virtio-pci

wateros-platform (components/wateros-platform/Cargo.toml)
- impl-qemu-riscv64-opensbi
- arch/impl-riscv64
  > wateros-platform-arch-impl-riscv64
- impl-qemu-loongarch64-virt
- arch/impl-loongarch64
  > wateros-platform-arch-impl-loongarch64

... # wateros-syscall、wateros-vfs 等

```

2.6.2 当前启用的 feature, config.conf

config.conf 记录并编辑当前实际使用的 feature: make configure 读取各 crate 现有 default, 把已启用项写成未注释行、未启用项写成 # 注释行。改配置、make apply_features、再编译, 才是切换组件 impl-* 的常规流程; 全量树里能看到但未在 config.conf 中启用的 feature, 不会进入当前构建。

- 未注释行表示启用; 行首 # 表示禁用。
- crate 名以冒号结尾, 例如 wateros-driver:; 其下缩进两格的是该 crate 可选 feature。
- 只导出非 api-v0/impl-* 命名的 workspace crate; *-api-v0 与 *-impl-* 叶子 crate 不出现在树中。
- 根 wateros: 下列出全部根 feature, 含 qemu-riscv64-opensbi 等平台组合; 组件 crate 下

列 `api-v0`、`impl-*` 等可替换项。驱动子系统允许同时启用多个 `impl-*`，例如块设备 MMIO 与 PCI 桩并存于配置树，实际生效仍受平台 feature 约束。

文件头与根 crate 片段示例如下，完整文件见 `os/config.conf`：

```
# WaterOS config.conf (只导出非 api/impl crate)
# 规则：未注释行 = 启用；以 # 开头 = 禁用。
# 结构：crate 行以 ':' 结尾；其下缩进两格的是 feature。

wateros:
  # qemu-loongarch64-virt
  qemu-riscv64-opensbi
  wateros-driver:
    api-v0
    wateros-driver-block:
      api-v0
      impl-virtio-mmio
      # impl-virtio-pci
  wateros-task:
    api-v0
    impl-core
    impl-multi-class
    # impl-round-robin
```

典型工作流：需要查看可选组合时读 `feature-tree.txt`；需要修改当前实现时编辑 `config.conf`。改配置前先 `make configure` 与 `manifest` 对齐；注释/取消注释目标 feature 后 `make apply_features` 写回 `default`，各 `Cargo.toml` 同目录会备份 `Cargo.toml.wosbak`，撤销用 `make revert_features`。

案例：把 `wateros-task` 下 `impl-round-robin` 取消注释、并注释掉 `impl-multi-class`，执行 `make apply_features` 后重新 `make rv_check`，task 调度器 `default` 会指向 `round-robin` 实现。切换 QEMU 平台时，在 `wateros:` 下注释当前 `qemu-riscv64-opensbi`、启用 `qemu-loongarch64-virt`，`apply` 后再 `make la_check` 验证 LoongArch 组合能否通过静态检查。日常 `make kernel-la` 仍显式传入 `qemu-loongarch64-virt`，与根 `default` 是否一致无关。

2.6.3 Makefile 与 feature 相关命令

`os/Makefile` 把平台 feature 写死在编译变量里，同时提供配置文件的生成、应用与回滚目标。相关用法如下：

表 2-3: Makefile 中与 feature 相关的目标

目标	作用
configure	扫描全部 Cargo.toml：生成全量 feature-tree.txt，并按当前 default 导出 config.conf。
apply_features	读取 config.conf 中启用的行，写回各 crate 的 default feature，并备份 .wosbak；此后构建按新选择编译组件。
apply_features_la	同 apply_features，但读取 config-loongarch64.conf，需自行维护该文件。
revert_features	从各目录的 Cargo.toml.wosbak 恢复 Cargo.toml。
kernel-rv / kernel-la	编译时分别传入 <code>-features qemu-riscv64-opensbi</code> 或 <code>-no-default-features -features qemu-loongarch64-virt</code> 。
rv_check / la_check	cargo check，feature 与上两行编译变量一致。
kernel-rv-ltp-glibc 等	在 RV/LA 编译 flag 上追加 <code>bringup-ltp-glibc-only</code> 或 <code>bringup-ltp-musl-only</code> 根 feature。

命令行案例：

```
cd os

# 1. 扫描 Cargo.toml：导出全量 feature 树 + 当前启用的 config.conf
make configure

# 2. 查传递关系 → 看 feature-tree.txt
# 改当前 impl 选择 → 编辑 config.conf，再 apply
make apply_features

# 3. 按 Makefile 绑定的平台 feature 编译 / 检查
make rv_check          # RISC-V, --features qemu-riscv64-opensbi
make kernel-rv
make la_check           # LoongArch, --no-default-features +
↳ qemu-loongarch64-virt
make kernel-la

# 4. 仅 LTP 长跑：追加 bringup-ltp-* 根 feature
make kernel-rv-ltp-glibc
make rv_qemu_run        # 仍使用 kernel-rv 产物

# 5. 放弃 apply 改动，恢复 Cargo.toml
make revert_features
```

说明：configure 不修改源码，只刷新全量树并同步 config.conf 与 manifest 的一致性。apply_features 只消费 config.conf，不改 feature-tree.txt。kernel-rv/kernel-la 的平台 feature 仍

由 Makefile 显式传入；config.conf 管的是组件级 default，调度器、块设备后端、日志级别等。二者叠加后才是某次 cargo build 实际链入的 impl。

2.7 QEMU 双架构适配与一致性

ISA 与板级差异收在 platform、driver 和 mm 实现层；进入 task、fs、vfs、ipc 和 syscall 之后，用户程序看到的 fd、进程和系统调用编号保持一致。

表 2-4: QEMU 双架构适配对比

适配点	RISC-V64 QEMU + OpenSBI	LoongArch64 QEMU virt
启动参数	OpenSBI 传入 a0/a1，分别解释为 hart id 与 DTB 物理地址。	固件传入 argc/argv/envp；envp 常携带 FDT 或平台信息指针。
固件与控制台	OpenSBI console、timer、reset 后端；runtime 经 impl-firmware-opensbi 接早期输出。	QEMUvirt 平台后端、StableCounter 和 LoongArch CSR；runtime 经 impl-firmware-console 接 UART。
trap 与上下文	arch-impl/impl-riscv64 提供 trap frame、trap.asm 和 switch.S。	arch-impl/impl-loongarch64 提供 trap frame、trap.S 和 switch.S。
页表与 TLB	MM 侧 Sv39；架构层用 satp token 切换地址空间并刷新 TLB。	MM 侧 LoongArch64 三级 4KiB 页表；写 CSR.PGDL 设根，invtlb 刷新 TLB。
设备总线	扫描 DTB 中 VirtIO-MMIO 节点；块设备与网卡走 MMIO 后端。	PCI 枚举 VirtIO 设备；块设备与网卡走 VirtIO-PCI 后端。
上层一致性	共享 abi/impl-linux-generic64、syscall/impl-kernel、task 生命周期、VFS fd 表、FS 注册表、ipc/all、cred 与 klog。平台差异在进入这些模块前被转换为统一接口，openat、read、clone、execve 等路径不按 ISA 各写一套。	

2.8 内核入口代码流程

os/src/main.rs 按 board feature 分别编译两个 kernel_main，共享“runtime 与 arch 先就绪，trap 在分页前注册，驱动与 FS 在地址空间建立后初始化，最后开中断进调度”的骨架。下面摘录体现双架构差异的关键片段，省略日志、烟测和网络初始化等实现细节。

RISC-V64 入口在解析引导参数后，先探测 DTB 再进入 runtime 初始化：

```
#[cfg(feature = "qemu-riscv64-opensbi")]
pub fn kernel_main(boot_arg0: usize, boot_arg1: usize) -> ! {
    let _boot_context = BootContext::from(BootArgs::new(boot_arg0,
        ↪ boot_arg1));
    driver::init_when_boot(boot_arg1);
    // runtime::console / klog / logging / heap ...
    platform::arch::init();
    task::init();
    crate::trap_handler::init();
    // mm::kernel_mm::init ...
    if driver::active_impl::init_after_boot().is_ok() {
        fs::init();
        crate::user_bringup_bus::run();
    }
    // enable timer interrupt ...
    task::run_first_task()
}
```

LoongArch64 入口参数不同，且在 MM 初始化前显式关闭固件遗留分页：

```
#[cfg(feature = "qemu-loongarch64-virt")]
pub fn kernel_main(_argc: usize, _argv: usize, envp: usize) -> ! {
    // runtime init ...
    platform::arch::init();
    driver::init_when_boot(envp);
    task::init();
    crate::trap_handler::init();
    platform::arch::paging::init_paging_disable_mmu();
    // mm::kernel_mm::init (usable_end_ppn capped) ...
    if driver::active_impl::init_after_boot().is_ok() {
        fs::init();
        crate::user_bringup_bus::run();
    }
    task::run_first_task()
}
```

两条路径都把 trap_handler 注册放在 mm::kernel_mm::init 之前，把 user_bringup_bus 放在驱动与 fs::init 成功之后；定时器与全局中断在 run_first_task 之前打开。完整调用顺序以 os/src/main.rs 为准。

第 3 章 模块实现

本章按内核组件划分：每个一级组件一节，说明它在用户态运行路径里承担什么、与相邻模块如何衔接。具体符号、数据结构和 feature 选路写在各节代码摘录的注释里，正文只做概览。

3.1 内核集成：异常与系统调用入口

用户态运行路径上的异常、中断与系统调用并不落在单个 `wateros-* crate` 内，而是由组合层在任务子系统初始化之后注册架构 `trap handler`，把汇编入口交出的 `trap` 原因分流到系统调用、内存管理、任务与平台子系统。下文各组件节只描述本 `crate` 提供的接口与实现；本节说明跨组件衔接。

热路径上，用户态系统调用经分派写回返回值；页错误先尝试写时复制或延迟映射，失败则向当前任务投递段错误信号；监督态定时器中断按与内核主循环一致的周期重装并驱动调度 `tick`。内核态不可恢复 `trap` 进入停机循环；用户态无法继续执行的 `fault` 在记录诊断后结束当前进程组。

```
match trap_cause {
    // 用户态 ecall: 读调用号与六个参数, 分派 syscall 并写回 a0
    // exec 成功时已替换整帧, 不推进 sepc; 其余 syscall 将 PC 前进一条指令宽度
    // rt_sigreturn 走独立恢复路径, 失败则杀进程组
    TrapCause::Exception(Exception::UserEnvCall) => {
        let ret = dispatch_syscall_from_trap(nr, args);
        if !exec_succeeded(nr, ret) {
            cx.add_user_pc(SYS CALL_INSN_BYTES);
            cx.set_syscall_ret(UserRet(ret));
        }
    }
    // 用户页错: 先 COW, 再 lazy 映射; 仍失败则 SIGSEGV, 必要时 kill 进程组
    TrapCause::Exception(Exception::InstructionPageFault)
    | TrapCause::Exception(Exception::LoadPageFault)
    | TrapCause::Exception(Exception::StorePageFault) => {
        if mm::kernel_mm::handle_user_page_fault(aspaces, addr, access) {
            return;
        }
        syscall::raise_current_signal(SIGSEGV);
    }
    // 监督态定时器: 重装 TIMER_REARM_MS, 递增 tick, 触发 timer_tick 与
    ↪ schedule_tick
    TrapCause::Interrupt(Interrupt::SupervisorTimer) => {
```

```

platform::timer::set_timer_after_ms(TIMER_REARM_MS);
syscall::timer_tick(cx.returns_to_user());
task::schedule_tick();
}
// 内核态页错与其余未预期 trap: fatal_kernel_trap 停机, 避免 sret 回到损坏
↪ PC
_ => { /* fatal_kernel_trap 或用户态信号/kill */ }
}

```

3.2 ABI 组件

wateros-abi 定义用户态与内核共享的系统调用 ABI: Linux 风格错误码、六个寄存器参数包、调用号抽象与负 `errno` 返回值编码。聚合 `src/lib.rs` 按 `feature` 挂载版本化契约与具体号表实现; 默认 `feature` 为空, 调用方须显式启用契约层及目标平台号表, 或仅链接占位桩做编译烟测。

主线平台 `feature` 经根内核传递 Linux 通用 64 位实现, 同时启用契约与活跃号表类型别名。系统调用分发与平台陷阱路径通过本组件读取参数包与符号化调用号; 号表中的常量只表示 ABI 约定, 是否与内核处理函数一一对应由 `syscall` 实现层决定。契约按关注点拆为返回值编码、错误码枚举、调用号 `trait` 与 C 布局参数包等子模块, 便于用户态桩代码与内核分发层共用同一套类型而不互相拖拽实现细节。

```

// abi/lib.rs: 号表 trait + 主线平台活跃别名 (需 impl-linux-generic64)
pub mod syscall_number {
    pub use api_v0::syscall_number::{SyscallNumber, SyscallNumberTable};
    #[cfg(feature = "impl-linux-generic64")]
    pub use impl_linux_generic64::LinuxGeneric64 as
        ↪ ActiveSyscallNumberTable;
}

```

```

// 陷阱帧六个通用寄存器 ↪ 固定布局参数包
pub mod syscall_args { pub use api_v0::syscall_args::*; }
pub mod errno { pub use api_v0::errno::*; }
pub mod user_ret { pub use api_v0::user_ret::*; }

```

3.3 基础类型组件

wateros-base 提供全内核共用的地址与标识薄类型, 以及单核环境下的简单同步容器。它不决定堆有多大、定时器多长, 只保证各处对物理地址、虚拟地址和 hart 编号的说法一致。配置数值单独放在同目录下的 `base-config` 子 crate 里, 避免同一常量在两套源码里各写一遍。

根内核和其它组件通常同时依赖类型层与配置层：前者管怎么表示一个地址，后者管页多大、syscall 最多几个参数、klog 环多长这类 bring-up 常数。真机或自定义内存布局应以设备树探测结果为准，配置里的 QEMU 区间只是回退默认值。

```
// base/lib.rs: 无 feature, 四模块
pub mod addr; // 物理/虚拟地址与页号包装
pub mod boot; // 设备树物理基址别名
pub mod cpu; // hart 标识
pub mod sync; // 单核 RefCell 包装 (多核须换锁)

// base-config: 按子系统分文件
// syscall: MAX_SYSCALL_ARGS = 6
// mm: KERNEL_HEAP_SIZE、QEMU virt RAM/MMIO 回退区间
// task: 定时器周期、时间片上限
// fs/ipc/klog: 管道容量、页缓存规模、日志环大小
```

3.4 凭证组件

wateros-cred 维护按任务索引的进程凭证侧表与生命周期钩子，对齐 Linux 凭证结构的常用子集。用户任务默认以 root 身份启动；fork 复制凭证，线程 clone 共享所有者引用计数，execve 后预留更新入口，首版为 no-op。任务子系统不依赖本组件；系统调用路径与 bring-up 调用点显式调用聚合门面，避免在任务控制块内硬编码身份模型。

根 Cargo.toml 默认同时启用契约与 root 实现。启用 root 实现时导出类型及 spawn、fork、共享、reap 等运行时 API，以及身份读写与能力查询；仅启用契约时无侧表，仅供类型共享。系统调用层的凭证读写经聚合门面转发；inode 访问权限检查尚未接入打开路径，bring-up 阶段能力查询恒为允许。侧表与任务 ID 解耦存储，便于日后在不改动任务 crate 的前提下替换凭证策略或接入更完整的 setuid 语义。

```
// cred/lib.rs: 生命周期 hook — spawn → fork/share → exec → reap
pub fn on_user_task_spawned(tid: TaskId) {
    ↪ active_impl::on_user_task_spawned(tid); }
pub fn fork_cred(parent: TaskId, child: TaskId) {
    ↪ active_impl::fork_cred(parent, child); }
pub fn share_cred(parent: TaskId, child: TaskId) {
    ↪ active_impl::share_cred(parent, child); }
pub fn on_exec(tid: TaskId) { active_impl::on_exec(tid); }

// bring-up: impl-root 阶段 has_cap / may_access_inode 恒 true
pub fn has_cap(cred: &ProcessCredentials, cap: Capability) -> bool {
    active_impl::has_cap(cred, cap)
}
```

3.5 设备驱动组件

驱动组件把板级探测、块设备、字符设备和网络设备收成内核能统一注册的入口。引导早期只记下设备树物理地址；内存子系统就绪后再扫描总线、握手 VirtIO 块卡与网卡，并把串口、RTC 等字符设备登记进全局表。探测失败只打日志，不阻断内核继续启动。

RISC-V 主线从设备树找 MMIO VirtIO 与 NS16550；LoongArch 主线走 PCIe 配置空间枚举 VirtIO-PCI，早期 UART 用固定 MMIO 窗口。块设备可选一层写穿缓存；网卡注册后由内置协议栈周期性轮询，再与用户态 socket 路径衔接。设备就绪后文件系统组件刷新设备目录，根卷探测才能看到块设备节点。当前 bring-up 仍以第一块盘、第一张网卡为主，多实例表已具备但未全面用到。

```
// driver 聚合：两步引导 + 平台 feature 选路
pub fn init_when_boot(dtb_pa: usize) {
    ↪ active_impl::init_when_boot(dtb_pa); }
pub fn init_after_boot() {
    // RISC-V: DTB → virtio-mmio blk/net + UART
    // LoongArch: PCIe ECAM → virtio-pci blk/net + 板级 UART
    // 末尾 devfs::refresh(), 失败仅 warn
}
// block: BlockDevice、register_block_device、可选 CachingBlockDevice
// network::stack: init / poll / TCP-UDP socket 句柄
// character: null、RTC 等内置桩
```

3.6 文件系统组件

文件系统组件负责块设备上的根卷读写，以及内核自带的设备目录和进程信息伪文件。启动时先根据驱动枚举刷新设备节点、在默认块设备上探测文件系统类型并登记实现，但此时还不挂载根卷；用户态 bring-up 再单独完成一次读写根卷挂载。辅助卷可以额外打开，不会替换全局根句柄。

设备目录把平台注册的块设备和字符设备映射成路径，供根卷探测和后续打开；进程信息目录按任务输出 status、maps 等测例常用字段，并允许上层注册命令行与可执行文件路径等回调。块格式实现默认走 vendored 的 ext4 库，也保留一套可选的旧路径，两套写实现不应同时启用。系统调用一般不直接碰这里的实现 crate，而是经虚拟文件系统桥接消费稳定 API。

```
// fs/lib.rs: init 只探测并 set_active, 不 mount
pub fn init() {
    devfs::active_impl::refresh();
    let dev_path = devfs::active_impl::default_root_block_path()?;
    let device = devfs::active_impl::lookup_block_device(dev_path)?;
```

```

    for imp in registered_fs_impls() {
        // 按静态表顺序 probe, 优先 RW 再 RO
        if let Ok(Some(kind)) = imp.probe(&device) { /*
            ↳ set_active_fs_impl */ break; }
        }
    }
    // bring-up 另调 rootfs::mount_default_root_rw()
    // FsImpl: name / probe / mount_ro / mount_rw

```

3.7 进程间通信组件

进程间通信组件把等待队列、管道、futex、共享内存和信号收成一组可按 feature 打开的子能力。系统调用层负责 Linux 参数解码与错误码；虚拟文件系统给管道两端分配 fd；内存组件在共享段附着后完成用户映射；任务与陷阱路径配合完成可中断睡眠和信号投递。本组件不定义陷阱约定，只对外提供稳定门面。

主线启用完整 feature 后，管道走环形缓冲，futex 与调度协作，共享内存维护段注册表，信号区分进程级与线程级状态。等待队列语义委托任务子系统，避免 IPC 依赖方直接耦合调度内部结构。顶层占位实现仍只做链接自检，真实契约在各子 crate 内演进。

```

// ipc/lib.rs: 默认 waitqueue; all 再挂 pipe / futex / shm / signal
#[cfg(feature = "pipe")]
pub mod pipe { pub use ::pipe::*; }
#[cfg(feature = "futex")]
pub mod futex { pub use ::futex::*; }
#[cfg(feature = "shm")]
pub mod shm { pub use ::shm::*; }
#[cfg(feature = "signal")]
pub mod signal { pub use ::signal::*; }
pub mod waitqueue { pub use ::waitqueue::*; }

```

3.8 内核日志组件

wateros-klog 维护内核可查询的消息环，与运行时控制台日志低耦合。内核及其它组件经 record 与 klog_! 宏写入带时间戳与调用方 ID 的记录；用户态经 syslog(2) 按传统 ASCII 行格式读取，优先级前缀加换行。环容量等常量来自基础配置 crate，实现层采用覆盖式环形缓冲，满时丢弃最旧记录而非阻塞写者。

根 Cargo.toml 默认启用契约、环形缓冲实现、平台单调时钟与任务 API。平台定时器提供纳秒时间戳；任务 API 在无调度上下文时以零作为调用方 ID。bring-up 阶段不检查 CAP_SYSLOG，任意进程可读写与清空环；主线初始化完成后写入固定问候语，便于 dmesg

类工具验证链路。与 syscall 层的对接集中在 syscall 子模块，内核其它路径只需调用 record 或宏即可。

```
// klog/lib.rs: 自动填充时间戳与 caller_id, 委托环实现追加
pub fn record(level: u8, facility: u8, text: &[u8]) -> AppendResult {
    let mut meta = KlogRecordMeta::new(
        ts_nsec_now(), 0, facility, level, KlogFlags::empty(),
        ↪ caller_id_now(),
    );
    record_with_meta(&mut meta, text)
}
```

```
// 宏路径: 栈上格式化缓冲 (上限 512 字节) → record
macro_rules! klog_info {
    ($($arg:tt)*) => {{
        let mut s = $crate::KlogFmtBuffer::new();
        let _ = write!(s, $($arg)*);
        let _ = $crate::record($crate::api::LOG_INFO,
            ↪ $crate::api::LOG_KERN, s.as_bytes());
    }};
}
```

3.9 内存管理组件

内存管理组件 wateros-mm 负责虚拟地址语义、页表实现、物理帧分配与用户态装载，对上暴露契约 API、帧分配器和内核 bring-up 入口，不把页表内部类型泄漏给依赖方。页大小固定为 4 KiB，与 RISC-V Sv39 常用叶子页一致；大页不在本阶段表达。

根内核在内存子系统初始化阶段建立内核页表与可用物理帧池；任务在 fork 与 exec 路径持用户地址空间句柄；系统调用层经契约处理内存映射、堆扩展与用户缓冲区访问；陷阱路径缺页时回调本组件的缺页与写时复制处理。默认 feature 仅链接占位页表，可编译但不装载 ELF、不 fork；真实 bring-up 须启用 Sv39 或 LoongArch64 实现，二者通过条件编译互斥。

物理帧默认走栈式 LIFO 分配器，带引用计数以支撑 fork 写时复制；统计信息可供 proc 类查询。NUMA 策略当前为单节点桩，满足 bring-up 系统调用返回值即可；帧分配器为单核实现，无 buddy 或 slab 分层。

```
// mm/lib.rs: api-v0 契约 + frame_allocator + 互斥 impl 选择
pub use api_v0 as api;
pub use frame_allocator;
```

```
#[cfg(feature = "impl-sv39")]
use impl_sv39 as active_mm_impl;
#[cfg(feature = "impl-loongarch64")]
use impl_loongarch64 as active_mm_impl;

pub use active_mm_impl::user_aspace; // LoadedElf::user_aspace_ptr
↪ 解析入口
```

```
// mm/lib.rs: trap 缺页与 COW 由 kernel_mm 门面转发
pub fn handle_user_page_fault(
    aspace_ptr: usize,
    fault_addr: usize,
    access: api_v0::mmap::PageFaultAccess,
) -> bool {
    // 惰性 mmap / 栈扩展 / brk 等按需映射
    let mut alloc = crate::frame_allocator::GlobalPhysFrameAllocator;
    crate::user_aspace::with_user_aspace_mut(aspace_ptr, |aspace| {
        MmapOps::handle_page_fault(aspace, &mut alloc,
            ↪ VirtAddr(fault_addr), access)
    })
    .unwrap_or(false)
}

pub fn handle_cow_fault(aspace_ptr: usize, fault_addr: usize) -> bool {
    // fork 后首次写触发写时复制
    impl_sv39::kernel_mm_impl::handle_cow_fault(aspace_ptr, fault_addr)
        .unwrap_or(false)
}
```

3.10 平台组件

平台层 wateros-platform 把指令集相关的陷阱、分页控制寄存器与时间计数，与板级或 QEMU 环境下的串口、定时器、复位等能力，组合成内核可直接调用的统一入口。架构侧只处理 CPU 可见的硬件原语；板级实现负责引导参数、早期控制台、deadline 定时器编程与关机重启。聚合 crate 在两层之上再提供定时器换算、引导期时间频率、墙上时钟等组合语义。

内核引导路径首先依赖本组件安装陷阱向量，并在进入用户态前完成架构初始化。运行时控制台输出、内存管理页表切换、任务上下文切换与陷阱帧保存，都从这里取得与 ISA 和板级相关的原语；具体硬件差异被限制在各自的实现 crate 内，上层模块看到的接口保持一致。根内核通过平台 feature 选定整条实现链；占位 feature 仅保证编译，不能用于真实启动。

QEMU 主线在 RISC-V 上走 OpenSBI 串口与 SBI 定时器，在 LoongArch 上走 MMIO UART 与 Constant Timer。若引导已从设备树探测到 timebase，须在首次使用定时器前注入频率；组合层还须在任务初始化附近注册内核陷阱分发入口，把定时器 tick、系统调用与缺页等业务接到上层。

```
// platform/lib.rs: active_impl 由 impl-qemu-riscv64-opensbi 等 feature
↪ 选定
#[cfg(feature = "impl-qemu-riscv64-opensbi")]
pub use impl_qemu_riscv64_opensbi as active_impl;

pub mod arch {
    pub use arch::*;
    #[inline]
    pub fn init() {
        arch_boot(); // 首次用户态 trap 前须安装向量
    }
}

pub mod timer {
    pub fn now_tick() -> PlatformTimerResult<ArchTimeTick> {
        arch::time::read_time_tick().map_err(PlatformTimerError::Arch)
    }

    pub fn set_timer_after(d: Duration) -> PlatformTimerResult<()> {
        // arch tick + time::frequency_hz 换算，再经 active_impl 编程
        ↪ deadline
        let now = now_tick()?.0;
        let hz = tick_hz()?.0;
        // ...
        set_timer_deadline_tick(ArchTimeTick(deadline))
    }
}
```

```
// platform-arch/lib.rs: RISC-V 与 LoongArch64 互斥
#[cfg(all(feature = "impl-riscv64", feature = "impl-loongarch64"))]
compile_error!("select only one platform-arch implementation");

pub fn arch_boot() {
    #[cfg(feature = "impl-riscv64")]
    impl_riscv64::init_trap();
    #[cfg(feature = "impl-loongarch64")]
    impl_loongarch64::init_trap();
}
```

3.11 运行时组件

运行时层 wateros-runtime 聚合 panic 处理、控制台输出、开发日志与内核堆分配等早期运行能力。根 crate 只做子模块再导出，不在此定义新的初始化语义；各子能力的行为由对应子 crate 的 feature 决定，根内核按引导顺序分别挂接 panic 入口、初始化分配器、注册日志器并打开控制台。

QEMU 主线默认走平台控制台路径，把打印宏接到平台早期串口；占位 feature 下任何实际写控制台会触发未实现 panic。开发日志经着色输出上屏展示，不写入内核日志环；需要同时上屏与持久化时，调用方须分别走日志宏与 klog 路径。日志级别由 trace 到 error 的一组 feature 择一，多开时取最安静档。

可选串口 feature 再导出 QEMUvirt UART 驱动，供伪 shell 等路径使用，与控制台路径分工明确、互不替代。panic 路径在着色打印后请求平台关机；控制台契约当前仅覆盖写侧，无输入 API。内核堆默认使用链表分配器，也可选用 TLSF 后端；分配路径带中断屏蔽与递归检测，接近高水位时会发出警告。

```
// runtime/lib.rs: 子能力以 runtime::console 等形式再导出
pub mod panic {
    pub use panic::panic_handler; // 供根 crate 挂接 #[panic_handler]
}
pub mod console {
    pub use console::*;
}
pub mod logging {
    pub use logging::*;
}
pub mod heap_allocator {
    pub use heap_allocator::*;
}
#[cfg(feature = "serial-uart-virt")]
pub mod serial {
    pub use ::runtime_serial::*;
}
```

```
// runtime-panic: 着色打印后请求平台关机
pub fn panic_handler(_panic_info: &core::panic::PanicInfo) -> ! {
    use console::{println, AnsiColor};
    if let Some(location) = _panic_info.location() {
        println!( "... Panicked at {}:{}. {}",
            location.file(), location.line(), _panic_info.message());
    }
    use platform::reset::{shutdown, PlatformResetReason};
```

```

while let Err(_e) = shutdown(PlatformResetReason::SystemFailure) {}
loop {}
}

```

3.12 系统调用组件

系统调用组件是用户态进入内核后的总接线板：陷阱路径把调用号和六个寄存器参数交进来，这里按 Linux 通用 64 位编号分派到文件、内存、任务、IPC 等子系统，并把内部错误收成负的 `errno` 返回值。同一层还导出定时器 tick、信号投递等与陷阱协作的钩子。

契约层描述有哪些语义槽位、默认未实现时返回什么；实现层持有按号分发表和各具体处理函数。号表里的常量只表示 ABI 对齐，是否与内核实现一一对应要分别核对。任务子系统不反向依赖本组件，保持依赖单向。

```

// trap 入口: SyscallArgs → 号表分发
pub fn dispatch_syscall_from_trap(syscall_nr: usize, syscall_args:
    ↪ SyscallArgs) -> isize {
    active_impl::dispatch_syscall_from_trap(syscall_nr, syscall_args)
}

// C ABI: 汇编陷阱返回路径可直接调用
#[unsafe(no_mangle)]
pub extern "C" fn __wateros_syscall_dispatch_current(
    syscall_nr: usize, arg0: usize, /* arg1..arg5 */
) -> isize {
    let args = SyscallArgs::from_regs([arg0, arg1, arg2, arg3, arg4,
        ↪ arg5]);
    dispatch_syscall_from_trap(syscall_nr, args)
}

```

3.13 任务管理组件

任务组件 `wateros-task` 组合任务语义类型、按 feature 选定的控制块布局与调度算法，向上提供内核与用户任务创建、主动让出、fork 与 exec、等待与进程回收等稳定 API。调度器决定何时运行谁，实现 core 决定单个任务长什么样；更换调度策略只改调度 crate，更换控制块字段只改实现 core。

内核在陷阱初始化与内存就绪后调用本组件初始化进程 registry 与调度器，随后进入首次任务切换。时钟 tick 驱动时间片轮转，阻塞与睡眠委托等待队列。fork 与 exec 与内存组件协作获取新地址空间；陷阱返回路径在用户任务与架构陷阱处理之间同步陷阱帧。系统调用层映射 Linux 错误码并调用本组件聚合 API，用户态返回值由 ABI 组件写入陷阱帧。

默认 feature 启用 API 契约、实现 core 与多类调度骨架；纯轮转实现与多类实现互斥，编译期强制二选一。bring-up 期有效调度策略仍为 OTHER 类路径，FIFO 与 RR 队列已有骨架但未完整抢占语义；当前为单核假设，无 per-CPU 就绪队列。

```
// task/lib.rs: 初始化顺序与薄封装
pub fn init() {
    active_impl::init_process_registry();
    active_impl::process_model_self_test();
    scheduler::init();
}

pub fn spawn_user_task_from_loaded_elf(loaded: &LoadedElf) -> TaskId {
    // MM 装载 ELF 后转换 UserTask 规格并入队
    spawn_user_task(user_task_from_loaded_elf(loaded))
}
```

```
// task/lib.rs: fork 与内存组件协作
pub fn fork_current(
    child_stack: usize,
    new_aspace_ptr: usize,
    new_satp: usize,
) -> Option<TaskId> {
    // new_aspace_ptr / new_satp 由 mm::kernel_mm::fork_user_aspace 提供
    let parent_pid = current_process_task_snapshot().map(|task|
        ↪ task.pid)?;
    let child_id = scheduler::create_fork_child(child_stack,
        ↪ new_aspace_ptr, new_satp)?;
    active_impl::with_process_registry(|registry| {
        let _ = registry.create_process_like_fork(parent_pid, child_id,
            ↪ address_space);
    });
    scheduler::enqueue_ready_task(child_id);
    Some(child_id)
}
```

3.14 工具组件

wateros-utils 预留给与平台弱耦合的通用小工具。#![no_std], 无 [dependencies], 也没有 api/impl/子 crate 分层。当前对外仅导出占位函数，供 crate 内单测烟测；根 wateros 以 utils 别名默认依赖，但主线内核代码尚未消费其公共 API。

src/asm/riscv/print_register.S 保留早期 UART 寄存器打印辅助，未编入现行构建。后续可在此集中纯函数或数据结构，并保持不反向依赖 wateros-base 等平台类型，以维持依赖

DAG 清晰。

```
// utils/lib.rs: 早期占位, 不代表最终公共 API
#[inline]
pub fn add(left: u64, right: u64) -> u64 { left + right }
```

3.15 虚拟文件系统组件

虚拟文件系统叠在文件系统组件与系统调用之间，把路径上的操作和每个进程手里的 fd 分开处理。它负责路径规范化、挂载表路由、普通文件的页缓存，以及每任务的工作目录和 fd 表；管道端点和部分字符设备也在这里落成可 read/write 的句柄。块格式与 ext4 细节不在这个组件里，由桥接层转调文件系统组件。

对用户程序来说，打开、读写、目录遍历和管道最终都落到本组件的门面；对下则把 ext4 根卷、设备目录和 proc 视图拼成统一后端。占位 feature 只保证链接通过，没有真实 I/O；主线启用桥接与 fd 会话实现后，才接上根卷读写和 per-task 状态。

```
// vfs 聚合: 契约 + 可选 FsBridge 后端 + fd 会话
pub mod active_impl {
    pub fn backend() -> &'static impl VfsBackend {
        // bridge-fs-api -> FsBridge; 否则 DummyBackend
    }
}
// vfs::root: 单根只读探测
// vfs::mount: 按种类打开 RW 会话
// impl-fd-session: stdin/stdout/stderr、动态 fd、pipe、cwd
// 桥接层: mount 表、页缓存句柄、tmpfs 伪卷、proc 回调注册
```

第 4 章 测试与问题处理

4.1 构建与启动方式

WaterOS 的内核构建与 QEMU 运行入口在 `os/Makefile`。日常操作固定为进入 `os/` 目录后执行 `make` 目标；`Makefile` 已封装 `target triple`、`feature` 组合、产物拷贝与底层脚本调用。

```
cd os

# 双架构内核产物 (make all = kernel-rv + kernel-la)
make all
make kernel-rv      # 产物 ./kernel-rv
make kernel-la      # 产物 ./kernel-la

# QEMU 运行 (目标会先触发编译)
make rv_qemu_run     # RISC-V, 块设备 sdcard-rv.img
make la_qemu_run     # LoongArch, 块设备 sdcard-la.img

# 日志与 snapshot (snapshot 不写回磁盘镜像)
make rv_qemu_run_with_log # QEMU -d int,cpu,in_asm → os/qemu.log
make rv_qemu_run_snapshot
make la_qemu_run_snapshot

# 静态检查 (feature 已与 Makefile 对齐)
make rv_check
make la_check
make check           # version + rv_check

# 从赛题目录刷新 ext4 根卷镜像
make flush_img

# PC 跳变监视与符号解析 (需 ADDR=)
make rv_pc_watch
make la_pc_watch
make rv_symbol_at ADDR=0x80201234
make la_symbol_at ADDR=0x90001234
```

RISC-V64 路径使用 `target riscv64gc-unknown-none-elf`，启用 `feature qemu-riscv64-opensbi`；LoongArch64 路径使用 `loongarch64-unknown-none`，以 `-no-default-features` 配合 `qemu-loongarch64-virt`。块设备镜像为 `os/sdcard-rv.img` 与 `os/sdcard-la.img`，由

os/scripts/rv_qemu_run.sh 或 la_qemu_run 内联 QEMU 参数挂载；RISC-V 脚本默认打开 virtio-net 与-netdev user，LoongArch 在 Makefile 中同样接入 virtio-net-pci。

4.2 功能验证

内核启动后先跑分层自检，再进入用户态 bring-up 总线。os/src/user_bringup_bus.rs 在 fs::init 之后挂载 RW ext4 根卷、确保/proc 并挂 procfs，随后登记 stage-busybox。赛题脚本队列在 user_bringup_busybox.rs 的 BRINGUP_COMMANDS 中维护：通过注释或取消注释 BringupCommand 行选择阶段，也可用 featurebringup-ltp-glibc-only/bringup-ltp-musl-only 单独长跑 LTP。

按 docs/tasks/run_testsuits_qemu.md，回归应分阶段串行，每次 QEMU 启动只启用一个阶段块，避免未实现 syscall 直接 panic 导致后续脚本不执行：

表 4-1: 赛题测例阶段 P1 至 P6

阶段	脚本, glibc 与 musl 各一份	主要覆盖	环境注意
P1	basic_testcode.sh	文件 IO、进程、内存 syscall	单盘时 mount 预期-22
P2	busybox_testcode.sh、lua_testcode.sh	shell 管道、TTYioctl、readv	busybox 须逐条 testcase ...success
P3	lmbench/unixbench/libcbench/getusage	消息、信号、多进程	unixbench 在代码中已注释弃用
P4	iperf_testcode.sh、netperf_testcode.sh	TCP/UDP 与后台服务	需 virtio-net
P5	libctest_testcode.sh、cyclictst_testcode.sh	动态链接、TLS、定时器	建议两组分开启用
P6	ltp_testcode.sh	最广 syscall 与 VFS 边界	体量大，单独长跑

阶段批跑可调用 os/scripts/run_phase_tests.sh，执行时会临时改写 BRINGUP_COMMANDS 并在结束后恢复。单次手动回归示例：

```
cd os

# 在 user_bringup_busybox.rs 中只取消注释目标阶段的 BringupCommand
make rv_qemu_run 2>&1 | tee /tmp/wateros_P1.log

# 按脚本块汇总 basic/busybox/libctest/ltp 等结果
python3 scripts/parse_qemu_test_log.py /tmp/wateros_P1.log
```

```
# PANIC 时提取 syscall 号
```

```
grep -nE 'unsupported: unknown nr=|Panicked at' /tmp/wateros_P1.log
```

表 4-2: 功能验证入口与日志特征

验证入口	覆盖内容	日志特征
启动期自检	MM 帧分配与页表探针、驱动扫描、FS 探测、网络栈 smoke。	[self-test] mm self-test done、driver init done、[fs::ext4][test] verify OK
bring-up 总线	RW 根卷、/proc、根布局硬链接修复。	[bringup][stage-00-bus]、[root-layout]
脚本 runner	ELF 装载、spawn、wait、脚本间	[busybox-bringup] program=、elapsed=、END path=...
测例输出	purge_all_user_processes。 各 *_testcode.sh 内 COMP TEST GROUP 与组内断言。	exit_code= #### OS COMP TEST GROUP START、testcase busybox、Pass!、FAIL LTP CASE
失败分类	panic、页故障、负 errno。	unsupported: unknown nr=、LoadPageFault、[syscall] nr=...ret=-

判读时不应只看 COMP TEST GROUP END: busybox 必须统计约 55 条 testcase busybox ...success; basic 看每个 Testing 后是否有 END test_; LTP 看 FAIL LTP CASE ...: 0。解析脚本 parse_qemu_test_log.py 按脚本块切分日志并汇总各组通过/失败计数, PANIC 时需额外 grep syscall 号对照 wateros-syscall 分发实现。

4.3 双架构一致性验证

RISC-V64 与 LoongArch64 分别编译、分别启动, 进入 MM、task、FS/VFS、IPC、cred、klog 与 syscall 后共用上层逻辑。验证重点是平台差异是否停在启动参数、trap frame、页表/TLB 与 VirtIO 总线层, 用户态 syscall 返回值与 errno 保持一致。

表 4-3: 双架构验证要点

验证点	RISC-V64 路径	LoongArch64 路径
-----	-------------	----------------

构建产物	make kernel-rv 生成 kernel-rv, 目标 triple 为 riscv64gc-unknown-none-elf。	make kernel-la 生成 kernel-la, 目标 triple 为 loongarch64-unknown-none。
启动入口	OpenSBI 传入 hart id 与 DTB; DTB 扫描 VirtIO-MMIO。	固件传入 argc/argv/envp; PCI 枚举 VirtIO-PCI。
内存路径	Sv39、satp、sfence.vma。	独立三级页表、CSR.PGDL、invtlb。
自检日志前缀	[self-test]	[loongarch64][self-test]、[driver-la]
用户态一致性	共用 Linux generic 64-bit syscall 编号、fd 表、cwd、VFS 路径、ELF 装载、fork/exec/wait、pipe、futex、procfs 与 klog; 相同脚本在两条路径上对比退出码与组内输出。	

4.4 遇到的问题 and 解决方法

测例覆盖面扩大后, 失败往往落在 syscall 语义、挂载路径、资源回收或双架构底层差异上。下面按主题说明典型现象、处理思路, 并给出当前代码中的关键片段。下列路径均相对于 os/。

4.4.1 系统调用语义和错误码

现象。libc 与 LTP 会校验 stat/fstat/statx 字段、mount 标志以及别名 syscall, 例如 faccessat2。占位返回值或静默忽略 unsupported flags 会在更深路径上变成错误语义; 号表未收录的 syscall 在 bring-up 期会走 unsupported: unknown nr= 直接 panic。

处理。已接入对象路径的调用从 VFS 元数据填 Linux 结构体; mount 按文件系统类型分支; 别名表未命中时返回-ENOSYS 而非成功占位。

fstat 从已打开 fd 取 metadata 示例如下:

```
// fstat(2): 将已打开文件的元数据写入用户 stat 缓冲
pub(crate) fn sys_fstat(args: SyscallArgs) -> UserRet {
    let fd = args.arg(0);
    let stat_ptr = args.arg(1);
    if stat_ptr == 0 {
        return UserRet::from_error(ErrNo::EFAULT);
    }
    // 标准输入/输出/错误与动态 fd 分界
    if fd < VFS_FIRST_DYNAMIC_FD && (fd > VFS_STDERR_FD || fd <
        ↪ VFS_STDIN_FD) {
```

```

        return UserRet::from_error(ErrNo::EBADF);
    }
    match vfs::fd::with_current_io(fd, |handle| {
        let meta = handle.metadata()?;
        let mut stat = fill_linux_stat(&meta, meta.size);
        stat_times::apply_stat(&meta, &mut stat);
        Ok(stat)
    }) {
        Ok(stat) => match copy_to_user_struct(stat_ptr, &stat) {
            Ok(()) => UserRet::from_success(0),
            Err(e) => UserRet::from_error(e),
        },
        Err(e) => UserRet::from_error(vfs_error_to_errno(e)),
    }
}

```

mount 对 proc/tmpfs 的分发:

```

// mount(2): 按 fstype 与 flags 分发到 VFS 挂载表
if fstype == "proc" {
    if vfs::is_proc_mounted_at(mount_point.as_str()) {
        return UserRet::from_error(ErrNo::EBUSY);
    }
    if let Err(e) = vfs::ensure_proc_mount_point() {
        return UserRet::from_error(vfs_error_to_errno(e));
    }
    return match vfs::mount_procfs_at(mount_point.as_str()) {
        Ok(()) => UserRet::from_success(0),
        Err(VfsError::Exists) => UserRet::from_error(ErrNo::EBUSY),
        Err(e) => UserRet::from_error(vfs_error_to_errno(e)),
    };
}
if fstype == "tmpfs" {
    // source 仅允许空或 "none"
    if source_ptr != 0 { /* copy_user_path_cstr; EINVAL if invalid */ }
    let readonly = flags & MS_RDONLY != 0;
    match vfs::mount_tmpfs_at(mount_point.as_str()) {
        Ok(()) => {}
        Err(VfsError::Exists) => return
            ↪ UserRet::from_error(ErrNo::EBUSY),
        Err(e) => return UserRet::from_error(vfs_error_to_errno(e)),
    }
    if readonly {

```

```

    return match vfs::remount_readonly_at(mount_point.as_str()) {
        Ok(()) => UserRet::from_success(0),
        Err(e) => UserRet::from_error(vfs_error_to_errno(e)),
    };
}
return UserRet::from_success(0);
}

```

别名 syscall 未实现时统一返回 ENOSYS，见 syscall-impl/impl-kernel/src/lib.rs:

```

// 别名号表：已实现的走具体 sys_*, 其余返回 -ENOSYS (不 panic)
fn dispatch_syscall_aliases(syscall_nr: usize, args: SyscallArgs) ->
↳ isize {
    if syscall_nr == SYS_FSTATAT {
        return sys::sys_fstatat(args).0;
    }
    if syscall_nr == SYS_STATX {
        return sys::sys_statx(args).0;
    }
    if syscall_nr == SYS_FACCESSAT2 {
        return sys::sys_faccessat2(args).0;
    }
    // setxattr/getxattr 等扩展属性槽位 ...
    match syscall_nr { /* 5..=16 */ _ => {} }
    let _ = args;
    abi::user_ret::UserRet::from_error(abi::errno::ErrNo::ENOSYS).0
}

```

4.4.2 文件系统和 VFS 路径

现象。basic 与 busybox 同时触及 ext4 根卷、procfs、tmpfs 与/dev 节点；若所有路径都落到同一读写句柄，df/ps 会出现/proc/mounts 缺失或权限 errno 混乱。

处理。bring-up 总线先 mount_default_root_rw，再挂/proc；mount 把 tmpfs、procfs、cgroup/securityfs 与 bind/remount 标志交给 VFS 挂载表，错误映射为 EINVAL/EBUSY 等 Linux errno。根卷布局脚本 user_bringup_root_layout 在总线之后补齐/bin、/etc/passwd 与 busybox 硬链接，减少 which、LTP 对路径的依赖。

bring-up 总线入口，见 src/user_bringup_bus.rs:

```

///! 用户态 bring-up 须在 fs::init 之后、fs::test 之前，
///! 且依赖已挂载的单一 RW 根卷视图。
pub fn run() {

```

```

info!("[bringup][stage-00-bus] BEGIN");
match fs::mount_default_root_rw() {
    Ok(()) => info!("[bringup][stage-00-bus] ext4 root mounted
        ↪ (RW)"),
    Err(err) => {
        // 根卷失败则跳过后续用户 ELF 阶段
        warn!("[bringup][stage-00-bus] mount root RW failed: {err:?}
            ↪ \
                — skip user ELF stages");
        info!("[bringup][stage-00-bus] END");
        return;
    }
}

match vfs::ensure_proc_mount_point() {
    Ok(()) => {},
    Err(err) => warn!("[bringup][stage-00-bus] ensure /proc dir
        ↪ failed: {err:?}"),
}

match vfs::mount_procfs_at("/proc") {
    Ok(()) => info!("[bringup][stage-00-bus] procfs mounted at
        ↪ /proc"),
    Err(vfs::api::VfsError::Exists) => {
        info!("[bringup][stage-00-bus] procfs already at /proc")
    }
    Err(err) => warn!("[bringup][stage-00-bus] mount procfs failed:
        ↪ {err:?}"),
}

info!("[bringup][stage-00-bus] END");
crate::user_bringup_root_layout::ensure_busybox_path_links();
// crate::user_bringup_basic::run_stage_basic(); // 按阶段注释开关
crate::user_bringup_busybox::run_stage_busybox();
}

```

4.4.3 并发、锁和资源回收

现象。 P3/P5/P6 长时间跑多进程脚本时，容易出现 zombie 残留、脚本间 fd/socket 泄漏，或调度器在关中断区内停留过久导致 timer 抢占异常。

处理。 调度器用 InterruptGuard 在触碰就绪队列时关中断并在 __switch 前恢复；每个脚本结束后 purge_all_user_processes 并调用 drop_reaped_task_runtime_resources 回收地址空间与 syscall 侧表。

调度器中断守卫，见 wateros-task/.../impl-round-robin/src/lib.rs：

```
// RAII：构造时关全局中断，drop 时恢复；包裹所有可能触碰就绪队列与 TCB
```

```

// 的调度路径。
struct InterruptGuard {
    state: ArchInterruptState,
}

impl InterruptGuard {
    fn new() -> Self {
        let state = arch::interrupt::read_global_interrupt_state()
            .expect("read global interrupt state for scheduler guard");
        arch::interrupt::disable_global_interrupt()
            .expect("disable global interrupt for scheduler guard");
        Self { state }
    }

    /// 在即将 __switch 且不会再回到本栈帧时调用：立刻恢复关中断前状态，
    /// 并用 forget 避免 Drop 二次恢复。
    fn release_before_switch(self) { /* restore + mem::forget(self) */ }
}

impl Drop for InterruptGuard {
    fn drop(&mut self) {
        arch::interrupt::restore_global_interrupt_state(self.state)
            .expect("restore global interrupt state for scheduler
                ↪ guard");
    }
}

```

脚本结束清理，见 src/user_bringup_common.rs:

```

// 串行执行单个 ELF/脚本：装载 → spawn → wait → reap
task::wait_for_task_exit(tid);
let exit_code = task::reap_exited_task(tid).map(|e| {
    drop_reaped_task_runtime_resources(&e); // 地址空间 + syscall 侧表
    e.exit_code
}).unwrap_or(-1);

// 脚本间清理残留用户进程，避免影响下一组测例
let (purge, stray_exited) = task::purge_all_user_processes();
for exited in &stray_exited {
    drop_reaped_task_runtime_resources(exited);
}
if purge.killed_tasks > 0 {
    warn!("[{log_tag}] script cleanup killed {} stray user task(s) \

```

```

        after path={elf_path}", purge.killed_tasks);
    }
    trace!("[{log_tag}] END path={elf_path} exit_code={exit_code}");

```

4.4.4 网络与双架构差异

现象。 P4 网络测例与 smoke 测试需要区分 VFS fd 与 socket fd；任务退出后若 socket 侧表未释放，后续 bind/connect 可能拿到陈旧状态。双架构差异集中在 trap 参数布局、页表切换与 VirtIO-MMIO/PCI，不应渗透到上层 VFS 语义。

处理。 socket_fd.rs 为每个任务维护独立的 SocketRef 映射与引用计数，release_task 在 owner 引用归零时整表删除；网络设备在 RISC-V 经 DTB、LoongArch 经 PCI 注册到同一 BlockDevice/网络抽象后，上层 socket syscall 共用实现。

socket fd 注册表核心，见 syscall-impl/impl-kernel/src/socket_fd.rs：

```

///! Socket fd → network SocketRef 映射表。
///! 因 VfsIoHandle 不支持向下转型，每个 socket fd 的共享状态在此独立维护。
struct SocketFdRegistry {
    maps: BTreeMap<TaskId, BTreeMap<usize, SocketRef>>,
    status_flags: BTreeMap<TaskId, BTreeMap<usize, usize>>,
    owners: BTreeMap<TaskId, TaskId>,          // CLONE_FILES 等共享 fd 表
    ref_counts: BTreeMap<TaskId, usize>,
}

fn release_task(&mut self, task_id: TaskId) {
    let owner = self.effective_owner(task_id);
    self.owners.remove(&task_id);
    let Some(count) = self.ref_counts.get_mut(&owner) else {
        self.maps.remove(&task_id);
        return;
    };
    *count = count.saturating_sub(1);
    if *count == 0 {
        // 引用归零：删除该 owner 下全部 socket fd 状态
        self.ref_counts.remove(&owner);
        self.maps.remove(&owner);
        self.status_flags.remove(&owner);
    }
}

```

第 5 章 总结与后续工作

5.1 工作总结与项目特色

WaterOS 已经形成从平台启动、trap、分页、任务调度到用户 ELF 执行的内核主路径,并在这条路径上接入文件系统、VFS、fd、procfs、pipe、futex、socket、凭证、klog 和 Linux generic 64-bit syscall 兼容层。RISC-V64 与 LoongArch64 分别处理启动参数、trap frame、页表 token、TLB 刷新和 VirtIO 总线差异,上层 MM、task、FS/VFS、IPC 和 syscall 逻辑保持复用。

项目实施上的主要特点如下:

- **组件化组织**。平台、驱动、MM、FS、VFS、task、syscall 等核心模块采用 API、实现和聚合导出的结构,根 crate 负责组合,具体能力留在组件内部。
- **双架构边界清晰**。架构差异集中在平台、页表、trap 和设备总线层,用户态可见的 fd、cwd、VFS、进程和 syscall 语义保持一致。
- **用户态路径连续**。用户 ELF 装载、地址空间切换、系统调用分发、fd 表、procfs、tmpfs、futex、socket 和文件系统读写已经连接到同一条运行路径。
- **诊断入口完整**。启动阶段保留 MM、驱动、FS、VFS、网络和用户 ELF 的分层日志,配合 QEMU 日志、PC 监视和符号解析工具定位 panic、死循环、unknown syscall 和用户程序退出问题。

5.2 后续完善方向

后续完善沿已跑通的用户态主路径展开,重点收紧 Linux 语义、长时间运行稳定性和双架构一致性。主要工作包括:

1. **补齐双架构同构验证**。RISC-V64 和 LoongArch64 已经共用上层 task、MM、FS/VFS、IPC 和 syscall 逻辑,后续重点检查动态链接、clone 参数、TLS、页表/TLB 刷新和 VirtIO-MMIO/VirtIO-PCI 差异,保证相同用户程序在两条路径上得到一致的返回值和错误码。
2. **细化进程、线程和信号语义**。在已有 fork、exec、wait、futex 和调度接口基础上,继续补齐线程组、clone3、实时信号、job control、robust futex 和更多调度策略相关语义,减少复杂用户程序对 Linux 行为差异的感知。
3. **增强文件系统写路径**。围绕 ext4_rs、tmpfs、VFS mount table 和 page cache,继续完善 fsync、rename/unlink、busy umount、多挂载实例和脏页写回,避免文件大小、目录项和页缓存状态在异常路径下不一致。

4. **收紧资源回收和失败回滚。**fd、socket、shm、task、page cache 和内核堆分配都已经进入用户态主路径，后续重点处理 OOM、dup/close 并发、socket 关闭、共享内存 attach 失败和任务退出时的回收顺序。
5. **完善网络和设备路径。**在 virtio-net、smoltcp、AF_UNIX 和 socket fd registry 的基础上，继续处理 TCP/UDP 长连接、loopback、后台 server、多进程 socket 生命周期，以及不同 QEMU 网络参数下的设备初始化差异。
6. **改进运行记录和定位工具。**保留 QEMU 串口日志、PC 监视、符号解析和日志解析工具，进一步统一 RISC-V64/LoongArch64 的运行记录格式，使 panic、死循环、unknown syscall、用户程序断言失败和资源泄漏都能更快定位到对应模块。

5.3 非本队来源说明

WaterOS 内核主体、组件划分、平台适配、驱动接入层、VFS 桥接、fd 会话、procfs/devfs 封装、socket fd 侧表、Linux generic 64-bit syscall 分发与 feature 组织均由本队实现。os/components/下各一级 crate 的 api-v0、impl-* 边界和根 wateros 聚合逻辑属于自研代码；第三方依赖仅作为底层库或协议栈引入，不替代上述接口设计。

直接引入的非本队 Rust 依赖通过 Cargo 声明；其中 ext4_rs 与 ext4plus 以 os/vendor/本地副本 vendoring，并在根 os/Cargo.toml 的 [patch.crates-io] 中固定路径，便于记录修改与来源边界。其余依赖由 cargo 从 crates.io 解析，版本以 os/Cargo.lock 为准。

表 5-1: 主要第三方依赖与用途

依赖	在本项目中的用途	引入方式	许可证，仓库内可见
ext4_rs	ext4 只读/读写根卷解析与块级访问，对应 impl-ext4-rs	os/vendor/ext4_rs + patch	MIT
ext4plus	ext4 备用实现路径，对应 impl-ext4	os/vendor/ext4plus + patch	MIT / Apache-2.0
smoltcp 0.12.0	内核侧 TCP/UDP 协议栈，配合 virtio-net 与用户 socket syscall	crates.io	见上游 crate
virtio-drivers 0.12.0	VirtIO-MMIO / VirtIO-PCI 块设备与网卡驱动封装	crates.io	见上游 crate
fdt 0.1.5	RISC-V / LoongArch QEMU 路径解析设备树	crates.io	见上游 crate

sbi-rt 0.0.3	RISC-V OpenSBI 运行时调用，含控制台与关机等	crates.io	见上游 crate
riscv 0.16.0	RISC-V CSR 与架构相关辅助	crates.io	见上游 crate
rlsf / linked_list_allocator (runtime-heap-allocator)	内核堆分配器后端	crates.io	见上游 crate
log、spin 等	日志门面、自旋锁等基础运行时支撑	crates.io	见上游 crate

5.4 AI 工具使用说明

本队在开发过程中使用 AI 辅助编程工具生成或改写部分内核代码。凡经 AI 参与完成的代码块，均在源码中以// 由 AI 完成、// 本模块代码由 AI 完成等注释标注；队伍在合并前对生成结果做编译、运行测例和语义核对，标注仅用于追溯来源，不表示未经审查即可交付。且为了项目符合我们最初设计的预期，在内核代码中，我们仅将 AI 生成代码用于了 Impl 中，具体的 struct, trait 定义，即 Api 系列文件夹下的代码以及通过原语组合出的模块的功能，这些设计和实现均由团队成员亲自完成。另需说明，在使用 AI 辅助 debug 的时候，我们使用 AI 批量插入了 log 信息，以方便我们进行调试。但目前依旧存在清理不完全的问题，其中有一部分 log 代码仍是 AI 生成的，但因为不影响核心功能，故此处不再做列举。

其次 AI 也同样适用于辅助我们完成本文档的编写，包括但不先于：（1）检索每个文件中的 AI 生成标记并汇总路径和行范围，并随机提取代码片段作为代表。（2）辅助完成文档内错误用语的勘误，文档正文在写完之后由 AI 进行了格式化和润色。（3）辅助完成文档内的一些重复性工作的自动化，如在构建文档项目时，通过 AI 完成目录结构初始化。且除此以外为了保证讲解视频效果我们使用了部分 AI 配音或语音校准。

具体投入应用的由 AI 生成的代码片段均标记在 docs/exports/ai-usage-inventory.tsv。根据按行统计，此处做的为未忽略空行的统计，AI 生成的代码约占本项目总代码的 **18.7%**，总行数来自 os/scripts/stat_contribute.bash 的统计。另外，本统计仅统计由 Claude Code、Codex、Cursor 这类工具直接修改文件生成的代码，未统计编辑器内的辅助补全和代码提示。以下为具体罗列，第三列的行范围表示该范围内存在 AI 生成的代码片段，并非该范围内全部都是 AI 生成的代码。

wateros-abi, 2 条

表 5-2: wateros-abi AI 标注条目，同文件已合并

序号	路径	行范围
----	----	-----

1	os/components/wateros-abi/abi-impl/impl-dummy/src/lib.rs	1-22
2	os/components/wateros-abi/abi-impl/impl-linux-generic64/src/lib.rs	13-383

路径：os/components/wateros-abi/abi-impl/impl-linux-generic64/src/lib.rs

行范围：13-383

```
...
pub struct LinuxGeneric64;

impl SyscallNumberTable for LinuxGeneric64 {
    // 文件与描述符
    const READ: SyscallNumber = SyscallNumber(63);
    ...
}
```

wateros-driver, 7 条

表 5-3: wateros-driver AI 标注条目, 同文件已合并

序号	路径	行范围
1	os/components/wateros-driver/driver-block/block-impl/impl-block-cache/src/manager.rs	1-44
2	os/components/wateros-driver/driver-block/block-impl/impl-dummy/src/lib.rs	1-21
3	os/components/wateros-driver/driver-character/character-impl/impl-dummy/src/lib.rs	1-18
4	os/components/wateros-driver/driver-character/character-impl/impl-null-stub/src/lib.rs	1-40
5	os/components/wateros-driver/driver-character/character-impl/impl-rtc-stub/src/lib.rs	1-56
6	os/components/wateros-driver/driver-impl/impl-dummy/src/lib.rs	1-18
7	os/components/wateros-driver/driver-network/network-impl/impl-dummy/src/lib.rs	1-51

路径：os/components/wateros-driver/driver-character/character-impl/impl-rtc-stub/src/lib.rs

行范围：1-56

```

///! 本模块代码由 AI 完成
///! 软件 RTC 字符设备 stub: `hwclock` 等通过 `/dev/misc/rtc` + `ioctl` 访问;
↪ 时间语义由 syscall 层处理。

#[no_std]

...

```

wateros-fs, 10 条

表 5-4: wateros-fs AI 标注条目, 同文件已合并

序号	路径	行范围
1	os/components/wateros-fs/fs-devfs/devfs-impl/impl-kernel/src/lib.rs	1-322
2	os/components/wateros-fs/fs-impl/impl-devfs/src/lib.rs	1-123
3	os/components/wateros-fs/fs-impl/impl-ext4-rs/src/lib.rs	1-829
4	os/components/wateros-fs/fs-impl/impl-ext4/src/boot_inspect.rs	1-77
5	os/components/wateros-fs/fs-impl/impl-ext4/src/lib.rs	1-97
6	os/components/wateros-fs/fs-impl/impl-ext4/src/ro.rs	1-291
7	os/components/wateros-fs/fs-impl/impl-ext4/src/rw.rs	1-971
8	os/components/wateros-fs/fs-impl/impl-ext4/src/selftest.rs	1-132
9	os/components/wateros-fs/fs-procfs/procfs-impl/impl-kernel/src/lib.rs	1-614
10	os/components/wateros-fs/fs-rootfs/rootfs-impl/impl-kernel/src/lib.rs	1-208

路径: os/components/wateros-fs/fs-impl/impl-ext4/src/rw.rs

行范围: 1-971

```

///! 读写路径 (基于 `ext4plus`, beta; 写路径无完整 journal, 仅用于 bring-up
↪ 与小文件测试)。
///!
///! I/O 边界: 块读写适配器将驱动错误装箱为 `ext4plus` 期望的 `Error` trait
↪ object; 按块读改写见本模块中的 `block_write_bytes`。
///! 本模块代码由 AI 完成

```

...

wateros-ipc, 6 条

表 5-5: wateros-ipc AI 标注条目, 同文件已合并

序号	路径	行范围
1	os/components/wateros-ipc/ipc-futex/futex-impl/impl-task/src/hub.rs	1-184
2	os/components/wateros-ipc/ipc-futex/futex-impl/impl-task/src/lib.rs	1-42
3	os/components/wateros-ipc/ipc-pipe/pipe-impl/impl-ringbuf/src/endpoint.rs	1-208
4	os/components/wateros-ipc/ipc-pipe/pipe-impl/impl-ringbuf/src/kernel_pipe.rs	1-438
5	os/components/wateros-ipc/ipc-pipe/pipe-impl/impl-ringbuf/src/lib.rs	1-75
6	os/components/wateros-ipc/ipc-waitqueue/waitqueue-impl/impl-task/src/lib.rs	1-187

路径: os/components/wateros-ipc/ipc-pipe/pipe-impl/impl-ringbuf/src/kernel_pipe.rs

行范围: 1-438

```
///! 固定容量 ring buffer pipe 实现。  
///! 本模块代码由 AI 完成
```

```
extern crate alloc;
```

...

wateros-klog, 9 条

表 5-6: wateros-klog AI 标注条目, 同文件已合并

序号	路径	行范围
1	os/components/wateros-klog/klog-api/api-v0/src/action.rs	1-52

2	os/components/wateros-klog/klog-api/api-v0/src/error.rs	1-22
3	os/components/wateros-klog/klog-api/api-v0/src/flags.rs	1-34
4	os/components/wateros-klog/klog-api/api-v0/src/lib.rs	1-17
5	os/components/wateros-klog/klog-api/api-v0/src/meta.rs	1-84
6	os/components/wateros-klog/klog-api/api-v0/src/store.rs	1-54
7	os/components/wateros-klog/klog-impl/klog-ringbuf/src/lib.rs	1-347
8	os/components/wateros-klog/src/export.rs	1-34
9	os/components/wateros-klog/src/lib.rs	1-165

路径：os/components/wateros-klog/klog-impl/klog-ringbuf/src/lib.rs

行范围：1-347

```

#![no_std]
///! desc 槽 + 每槽变长正文（上限 `KLOG_MAX_RECORD_BYTES`）的环形 klog 实现。
///! 本模块代码由 AI 完成

use api_v0::{
    ...

```

wateros-mm, 13 条

表 5-7: wateros-mm AI 标注条目，同文件已合并

序号	路径	行范围
1	os/components/wateros-mm/mm-impl/impl-dummy/src/lib.rs	1-54
2	os/components/wateros-mm/mm-impl/impl-loongarch64/src/kernel_executable.rs	1-65
3	os/components/wateros-mm/mm-impl/impl-loongarch64/src/lib.rs	96-143
4	os/components/wateros-mm/mm-impl/impl-loongarch64/src/pagetable.rs	234-773
5	os/components/wateros-mm/mm-impl/impl-loongarch64/src/user_access.rs	1-129
6	os/components/wateros-mm/mm-impl/impl-loongarch64/src/user_aspace.rs	1-29

7	os/components/wateros-mm/mm-impl/impl-loongarch64/src/user_heap_mmap.rs	1-554
8	os/components/wateros-mm/mm-impl/impl-sv39/src/kernel_executable.rs	1-73
9	os/components/wateros-mm/mm-impl/impl-sv39/src/lib.rs	98-144
10	os/components/wateros-mm/mm-impl/impl-sv39/src/pagetable.rs	252-865
11	os/components/wateros-mm/mm-impl/impl-sv39/src/user_access.rs	1-170
12	os/components/wateros-mm/mm-impl/impl-sv39/src/user_aspace.rs	1-27
13	os/components/wateros-mm/mm-impl/impl-sv39/src/user_heap_mmap.rs	1-555

路径: os/components/wateros-mm/mm-impl/impl-sv39/src/pagetable.rs

行范围: 252-865

```
...
pub(crate) struct LazyFileVma {
    pub start : VirtAddr,
    pub end : VirtAddr,
    pub perm : PagePerm,
    pub file_offset : usize,
    ...
}
```

wateros-platform, 7 条

表 5-8: wateros-platform AI 标注条目, 同文件已合并

序号	路径	行范围
1	os/components/wateros-platform/platform-arch/arch-impl/impl-dummy/src/lib.rs	1-20
2	os/components/wateros-platform/platform-impl/impl-dummy/src/lib.rs	1-109
3	os/components/wateros-platform/platform-impl/impl-qemu-loongarch64-virt/src/lib.rs	20-84

4	os/components/wateros-platform/platform-impl/impl-qemu-riscv64-opensbi/src/console.rs	1-31
5	os/components/wateros-platform/platform-impl/impl-qemu-riscv64-opensbi/src/lib.rs	22-76
6	os/components/wateros-platform/platform-impl/impl-qemu-riscv64-opensbi/src/reset.rs	1-90
7	os/components/wateros-platform/platform-impl/impl-qemu-riscv64-opensbi/src/timer.rs	1-17

路径：os/components/wateros-platform/platform-impl/impl-dummy/src/lib.rs

行范围：1-109

```

#![no_std]

///! 本模块代码由 AI 完成
///! ** 占位 ** 平台实现：启动参数与时间频率 trait 的桩，用于未绑定真实环境的构建。
///!
...

```

wateros-task, 2 条

表 5-9: wateros-task AI 标注条目，同文件已合并

序号	路径	行范围
1	os/components/wateros-task/task-scheduler/scheduler-impl/impl-multi-class/src/rt_fifo_queue.rs	1-154
2	os/components/wateros-task/task-scheduler/scheduler-impl/impl-multi-class/src/rt_rr_queue.rs	1-243

路径：os/components/wateros-task/task-scheduler/scheduler-impl/impl-multi-class/src/rt_rr_queue.rs

行范围：1-243

```

///! 本模块代码由 AI 完成
///! `SCHED_RR` 就绪队列：优先级桶 + 同优先级时间片轮转。

extern crate alloc;

```

...

wateros-utils, 1 条

表 5-10: wateros-utils AI 标注条目, 同文件已合并

序号	路径	行范围
1	os/components/wateros-utils/src/lib.rs	1-22

路径: os/components/wateros-utils/src/lib.rs

行范围: 1-22

```
#![no_std]
///! 通用小工具与可复用例程的聚合 crate (当前为早期占位)。
///!
///! 后续可在此集中与内核其它组件无强耦合的纯函数或数据结构;
///! 避免反向依赖 `wateros-base` 等平台类型, 以保持依赖 DAG 清晰。
...
```

wateros-vfs, 15 条

表 5-11: wateros-vfs AI 标注条目, 同文件已合并

序号	路径	行范围
1	os/components/wateros-vfs/vfs-impl/impl-fd-session/src/char_dev_handle.rs	1-371
2	os/components/wateros-vfs/vfs-impl/impl-fd-session/src/cwd.rs	1-174
3	os/components/wateros-vfs/vfs-impl/impl-fd-session/src/file_lock.rs	1-578
4	os/components/wateros-vfs/vfs-impl/impl-fd-session/src/handles.rs	1-596
5	os/components/wateros-vfs/vfs-impl/impl-fd-session/src/lib.rs	1-25
6	os/components/wateros-vfs/vfs-impl/impl-fd-session/src/registry.rs	1-781

7	os/components/wateros-vfs/vfs-impl/impl-fs-bridge/src/dir_handle.rs	1-166
8	os/components/wateros-vfs/vfs-impl/impl-fs-bridge/src/file_handle.rs	1-325
9	os/components/wateros-vfs/vfs-impl/impl-fs-bridge/src/lib.rs	1-1089
10	os/components/wateros-vfs/vfs-impl/impl-fs-bridge/src/mount_ns.rs	1-141
11	os/components/wateros-vfs/vfs-impl/impl-fs-bridge/src/mount_table.rs	1-894
12	os/components/wateros-vfs/vfs-impl/impl-fs-bridge/src/paged_handle.rs	1-611
13	os/components/wateros-vfs/vfs-impl/impl-fs-bridge/src/proc_handle.rs	1-209
14	os/components/wateros-vfs/vfs-impl/impl-fs-bridge/src/tmpfs.rs	1-702
15	os/components/wateros-vfs/vfs-impl/impl-page-cache/src/lib.rs	1-1115

路径：os/components/wateros-vfs/vfs-impl/impl-page-cache/src/lib.rs

行范围：1-1115

```

///! 全局共享文件页缓存 (Direct 模式)：按 `(mount_gen, path, page_index)` 键
↪ LRU 缓存页帧。
///!
///! 每个文件条目使用 [`Arc<RwLock<FileEntryInner>>`]：允许多个读者并发，
↪ 写/刷盘独占。
///!
///! ## Lock ordering
...

```

除上述标注块外，还有两类未逐条列入下表的 AI 辅助：一是模块层级的 `///!` 文档。队员对 cargo doc/rustdoc 编写规范不够熟悉，因此在保留原意的前提下，用 AI 将原先偏口语的模块说明整理为符合惯例的 crate/module 文档格式。二是 os/scripts/目录下的部分 QEMU 运行、日志解析、符号定位等辅助脚本，由 AI 参与编写或改写；具体文件以该目录内源码与版本记录为准。